

CAPSTONE PROJECT TECHNICAL REPORT

Automated Wealth Management Intelligence Platform

A Rigorous Engineering Study of Relational Data Architecture, Quantitative Risk Modelling, and Automated ETL Orchestration

Field	Value
Document Type	Academic-Grade Capstone Technical Report
Domain	Financial Data Engineering · Quantitative Risk Analytics
Classification	Confidential — Academic Submission
Version	1.0 — Final
Date	June 2026

1. Executive Summary

1.1 Detailed Project Overview

The global wealth management industry stands at an extraordinary inflection point, characterised by the rapid convergence of institutional-grade quantitative finance, cloud-native data engineering paradigms, and artificial intelligence-driven advisory models. In an era where total assets under management (AUM) across mutual funds, portfolio management services (PMS), and alternative investment funds (AIFs) routinely surpass multi-trillion-dollar benchmarks, the operational infrastructure underpinning these assets has, paradoxically, remained archaic. Many fund houses still rely on manually curated spreadsheets, siloed reporting databases, and bespoke analytical tools that lack both the robustness and the reproducibility demanded by modern regulatory environments and sophisticated institutional investors.

This capstone project directly confronts that institutional gap by constructing an end-to-end, automated wealth management intelligence platform that integrates three mutually reinforcing engineering disciplines. First, a rigorously normalised relational data layer — implemented in a lightweight, portable SQLite engine — serves as the single source of truth for all fund-level data, establishing formal entity relationships through primary and foreign key constraints that guarantee referential integrity across the schema. Second, a multi-stage Extract, Transform, and Load (ETL) pipeline extracts raw records from this structured relational store, applies systematic data-type coercion, temporal index construction, and schema validation checks, before materialising clean, analytics-ready data frames into the processing memory layer. Third, and perhaps most critically from a financial economics perspective, a suite of non-parametric quantitative risk models — specifically, Historical Simulation Value at Risk (VaR) and Conditional Value at Risk (CVaR) — translates the cleaned transactional time-series data into actionable, statistically defensible down-market risk metrics.

The intersection of these three pillars — structured data governance, automated engineering orchestration, and empirical risk quantification — is not merely an academic exercise. It represents the foundational architecture that modern wealth management firms must adopt in order to meet the twin demands of regulatory scrutiny and investor transparency. Regulatory frameworks such as SEBI's Risk-O-Meter guidelines for mutual funds, ESMA's UCITS risk categorisation requirements in European markets, and the SEC's investment adviser risk disclosure mandates all implicitly require that fund managers possess the technical capability to compute and communicate portfolio risk at scale, in near-real time, with full methodological auditability. The platform developed in this project is designed precisely to satisfy these requirements, delivering a reproducible, modular, and extensible architecture that can evolve from an academic prototype into a production-grade enterprise system.

At its core, the project reflects a philosophical commitment to the idea that superior wealth management outcomes are not achieved through superior stock-picking intuition alone, but through superior information architecture — the kind of architecture that ensures every layer of the data ecosystem, from raw transaction records to the final risk disclosure report, is governed by explicit rules, validated at every stage, and reproducible on demand.

1.2 Comprehensive Problem Statement

The fundamental failure mode of conventional wealth management reporting systems resides in their almost exclusive reliance on trailing performance metrics — metrics that are, by mathematical construction, backward-looking and incapable of providing credible predictive signals during periods of market stress. Return metrics such as the three-year Compound Annual Growth Rate (CAGR), Sharpe Ratio, and Sortino Ratio are computed over historical windows and implicitly assume that the distributional properties of the return series — most critically, the mean and variance — remain stationary across the measurement horizon. This assumption is not merely intellectually inconvenient; it is

empirically false. Extensive econometric literature, including the foundational work of Mandelbrot (1963) on heavy-tailed asset return distributions and the volatility clustering models of Engle (1982) in the ARCH framework and its generalisations by Bollerslev (1986), demonstrates unambiguously that financial return series exhibit pronounced non-stationarity, particularly during market contractions.

During a market contraction — defined operationally as a period of sustained negative returns across multiple asset classes, typically associated with elevated systemic risk premia, widening credit spreads, and deteriorating liquidity conditions — trailing metrics become actively misleading. A fund that achieved a 15% three-year CAGR through aggressive equity exposure in a bull-market regime will present an attractive risk-adjusted return profile on paper even as its portfolio concentrations render it acutely vulnerable to rapid drawdown when regime shifts materialise. The Sharpe Ratio, in particular, suffers from its normalisation against the standard deviation of returns, which understates tail risk in distributions exhibiting negative skewness and excess kurtosis — precisely the distributional characteristics that emerge during financial crises. Investors and advisors who rely exclusively on these trailing metrics therefore operate with a systematically distorted perception of portfolio risk, one that is most dangerously misleading precisely when accurate risk assessment matters most.

In parallel, the technical challenges confronting data engineers who attempt to build robust analytics infrastructure for financial data are formidable and largely underappreciated in the academic literature. The most pervasive of these is schema instability — a phenomenon whereby the structural definition of a relational table or data source evolves over time in ways that are not formally documented, not backwards-compatible, and not detected by the pipeline until a downstream failure has already corrupted an analytics output. Schema drift, as it is termed in production data engineering, can manifest in numerous forms: a new column is added to a source system's export without notification; an existing column's data type is silently changed from a string to a floating-point numeric; a mandatory foreign key field begins returning null values due to an upstream system configuration change; or an enumerated field begins accepting new categorical values that are not recognised by downstream classification logic. Each of these events, if undetected, propagates through the pipeline and produces silently incorrect analytical outputs — a failure mode that is in many respects more dangerous than an overt pipeline crash, because it poisons the decision-making process without triggering any visible alarm.

The relational financial data schema constructed in this project addresses schema drift through a combination of strict field-level data type casting, explicit null-value detection logic, and primary key uniqueness enforcement — a set of controls collectively described as schema drift mitigation measures. These controls ensure that any deviation from the expected data contract is detected at the ingestion boundary of the ETL pipeline and flagged for remediation before the corrupted data can propagate into the analytics layer. This approach represents a meaningful departure from the common practice of accepting raw data as-is and deferring data quality validation to the analytical layer, where the cost of remediation is substantially higher.

1.3 High-Level Business Outcomes

The organisational value generated by a fully automated, rigorously engineered wealth management intelligence platform extends far beyond the immediate technical deliverables of a cleaner database or a more accurate risk metric. When a master orchestration script — in this project instantiated as the `run_pipeline.py` execution controller — assumes responsibility for coordinating every step of the data processing workflow, the operational risk associated with human intervention in the data pipeline is systematically eliminated. Every manual data pull, every ad hoc transformation applied in a spreadsheet, and every informal data quality check performed by an analyst represents a potential point of failure and a source of undocumented, irreproducible decision-making. Automation converts these informal, error-prone processes into formal, version-controlled, auditable code that executes identically on every run, irrespective of which team member triggers it or what time of day it runs.

The deployment of empirically calibrated down-market risk buffers — specifically the 95% VaR and CVaR metrics computed in this platform — provides risk and compliance teams with quantitative thresholds that can be directly translated into client suitability assessments, regulatory risk disclosures, and portfolio rebalancing triggers. Rather than relying on qualitative judgements about a fund's risk character, relationship managers and investment advisors can now anchor their client conversations in statistically defensible numbers: this particular equity scheme has a 95% VaR of -1.28%, meaning that in the historical distribution of daily NAV movements, only 5% of days produced losses greater than 1.28%. This is language that sophisticated investors understand and can incorporate into their own risk frameworks.

The dynamic system architecture — characterised by its modular, layered design with clearly demarcated responsibilities at each processing stage — provides the organisation with strategic optionality. Because the data ingestion layer, the transformation layer, and the analytics layer are decoupled from one another, each can be upgraded, replaced, or horizontally scaled independently, without requiring a full-system redesign. This architectural characteristic is of particular commercial significance as the organisation's data volumes grow, as new fund categories are added to the analytical scope, or as regulatory requirements evolve to mandate additional risk metrics. The platform is engineered not merely to solve today's problem, but to remain relevant and adaptable as the problem set evolves.

2. Relational Data Layer & Source Framework

2.1 Database Environment

The foundational data persistence layer of this platform is implemented using SQLite, a self-contained, serverless, zero-configuration relational database engine that stores the entire database schema, data, and indexing structures within a single binary file on the host filesystem. The selection of SQLite over more heavyweight relational database management systems — such as PostgreSQL, MySQL, or Microsoft SQL Server — was driven by a set of architectural considerations specific to the project's context and objectives. In a research and prototyping environment where the primary concern is rapid iteration, environmental portability, and elimination of infrastructure dependencies, SQLite's embedded nature provides decisive advantages. There is no daemon process to manage, no TCP/IP connection pool to configure, no authentication scheme to maintain, and no installation procedure beyond including the `sqlite3` library in the runtime environment — a library that ships natively with all major Python distributions.

From a storage architecture standpoint, SQLite employs a page-based B-tree storage model in which the entire database is segmented into fixed-size pages, typically 4096 bytes in the default configuration, that are managed by the SQLite page cache. Each table in the database is stored as a separate B-tree structure, with indexed columns maintained as secondary B-trees that store pointers back to the primary table pages. This architecture provides $O(\log n)$ lookup complexity for indexed queries, which is more than adequate for the dataset sizes encountered in this project — historical NAV time series spanning several years across dozens of funds typically encompass hundreds of thousands of records at most, well within the range where SQLite's single-writer concurrency model presents no meaningful operational constraint.

The structural configuration of the SQLite database enforces referential integrity through the explicit activation of foreign key constraint checking — a feature that is, counterintuitively, disabled by default in SQLite for backwards compatibility reasons and must be explicitly enabled at the beginning of each database connection session using the `PRAGMA foreign_keys = ON` directive. Without this directive, `INSERT` and `DELETE` operations on related tables would proceed without validating the consistency of cross-table relationships, potentially allowing orphaned records to accumulate in fact tables — a form of silent data corruption that would undermine the analytical reliability of the entire system. The explicit activation of this pragma is therefore a non-negotiable configuration step, embedded in the database connection initialisation logic of the ETL pipeline.

The database file is stored on the local filesystem of the application host, which confers both advantages and limitations. The primary advantage is speed: local disk I/O eliminates the network round-trip latency that characterises all client-server database architectures, and modern solid-state storage devices can sustain sequential read throughputs of several gigabytes per second, making full-table scans of even large fact tables operationally trivial. The primary limitation is the absence of concurrent multi-user write access — SQLite supports only one writer at a time, with all other write operations blocked during an active transaction. For the single-user, batch-processing architecture of this project, this constraint is operationally irrelevant. For a production deployment serving multiple concurrent analytical users or real-time data ingestion streams, it would represent a meaningful bottleneck requiring architectural escalation to a client-server database engine.

2.2 Detailed Entity Schema Definitions

The relational schema of this platform is structured around three core entities: `dim_fund`, `fact_nav`, and `fact_transactions`. These entities embody the canonical dimensional modelling pattern pioneered by Ralph Kimball, in which slowly-changing descriptive attributes are separated into dimension tables (prefixed `'dim_'`) while transactional and measurement data are stored in fact tables (prefixed `'fact_'`). This

structural separation not only mirrors best practice in enterprise data warehousing but also enforces a clean conceptual boundary between what a fund is — its identity, classification, and regulatory characteristics — and what a fund does — its daily price movements and investor transaction activity.

2.2.1 The dim_fund Dimension Table

The dim_fund table functions as the master reference entity for all fund-level descriptive attributes within the platform. Its primary key, fund_id, is defined as a non-null integer surrogate key that is auto-incremented by the database engine upon each new fund record insertion. The use of a surrogate key — an arbitrary integer identifier with no inherent business meaning — rather than a natural key such as the fund's AMFI code or ISIN number is a deliberate architectural choice. Natural keys drawn from external systems are subject to change due to external administrative decisions entirely outside the control of the local system; a fund's ISIN, for example, may be re-issued following a fund merger or scheme restructuring. The surrogate fund_id, by contrast, is entirely under local system control and is guaranteed to remain stable for the lifetime of the record, providing a dependable anchor for all foreign key references from fact tables.

Beyond its primary key, dim_fund stores a set of descriptive attributes that define the fund's regulatory classification and analytical context. The fund_name field stores the complete legal name of the scheme as registered with the relevant securities regulator. The fund_category field stores a standardised categorical label — such as 'Equity: Large Cap', 'Equity: Multi Cap', 'Debt: Short Duration', or 'Hybrid: Aggressive' — drawn from a controlled vocabulary that aligns with the fund categorisation taxonomy mandated by SEBI's October 2017 circular on scheme rationalisation, which defines 36 permissible mutual fund categories in the Indian market. The risk_level field stores an ordinal risk classification — typically a value from 1 to 7 corresponding to the SEBI Risk-O-Meter scale — that provides a coarse regulatory risk bucketing for the fund. Together, these attributes enable the analytical layer to perform group-level risk aggregation, comparing VaR and CVaR distributions across fund categories and risk levels.

2.2.2 The fact_nav Fact Table

The fact_nav table is the central time-series measurement entity of the platform, recording the daily Net Asset Value — the per-unit price of the fund computed as the total value of the fund's assets minus its liabilities divided by the total number of outstanding units — for each fund in the dim_fund dimension. Its composite primary key consists of the combination (fund_id, nav_date), which enforces the domain constraint that each fund can have at most one NAV record per calendar day. The fund_id column simultaneously functions as a foreign key referencing the fund_id primary key of dim_fund, establishing a formal one-to-many relationship between the dimension and fact tables: a single fund record in dim_fund can be associated with thousands of daily NAV records in fact_nav, but every NAV record must be associated with exactly one valid fund record.

The nav_date column is stored as a TEXT field in ISO 8601 format (YYYY-MM-DD), which is the canonical date representation recommended by SQLite's official documentation for temporal data. Because SQLite lacks a native DATE data type, all date arithmetic and sorting operations in SQL queries rely on the lexicographic ordering properties of ISO 8601 strings, which correctly reflect chronological ordering for well-formed date strings. The nav_value column stores the per-unit NAV as a REAL (double-precision floating-point) value. The choice of floating-point rather than fixed-point decimal storage introduces a potential precision artefact — floating-point representation cannot exactly represent many decimal fractions, and rounding errors can accumulate over large numbers of arithmetic operations. In the context of NAV time-series analytics, where the primary computation is the log-difference of successive NAV values, this precision limitation is operationally negligible: the rounding error introduced by double-precision floating-point is on the order of 10^{-15} , many orders of magnitude smaller than the meaningful precision of a NAV value quoted to four decimal places.

2.2.3 The fact_transactions Fact Table

The fact_transactions table records the economic events generated by investor activity — purchases (subscriptions), redemptions, and systematic investment plan (SIP) instalments — at the level of the individual transaction. Its primary key is a surrogate transaction_id integer, auto-incremented on each insert. Its foreign key references include fund_id (linking to dim_fund) and, in extended schema configurations, investor_id (linking to a dim_investor table that can be added without schema disruption due to the modular design of the relational model). Key attributes of this table include transaction_date (ISO 8601 text), transaction_type (a controlled vocabulary field accepting the values 'BUY', 'SELL', and 'SIP'), units (the number of fund units involved in the transaction, stored as REAL), and nav_at_transaction (the NAV per unit prevailing at the time of the transaction, stored as REAL), from which the total transaction value can be derived as a computed column.

The structural relational integrity of the schema as a whole — the network of primary key uniqueness constraints, foreign key referential constraints, and domain constraints — provides a mathematically precise definition of the set of valid states the database can occupy. Any INSERT, UPDATE, or DELETE operation that would produce a state outside this set is rejected by the database engine before it can execute, preventing the accumulation of structural inconsistencies that would otherwise silently corrupt downstream analytical outputs. This is the fundamental value proposition of a relational database over a flat-file storage model: not merely that data is stored in a structured format, but that the storage engine itself enforces the rules of data validity as a first-class operational concern.

Table 2.1 — Core Schema Entity Summary

Entity	Primary Key	Foreign Key	Key Attributes
dim_fund	fund_id (AUTOINCREMENT)	—	fund_name, fund_category, risk_level
fact_nav	(fund_id, nav_date)	fund_id → dim_fund	nav_date (ISO 8601), nav_value (REAL)
fact_transactions	transaction_id (AUTOINCREMENT)	fund_id → dim_fund	transaction_date, type, units, nav_at_transaction

3. Robust ETL Design & Orchestration Pipeline

3.1 Architectural Flow

The ETL pipeline of this platform is architecturally structured as a directed acyclic graph (DAG) of data transformation stages, each receiving a well-defined input data structure and producing a well-defined output data structure, with all inter-stage dependencies flowing in a single direction from raw source data to refined analytical output. This DAG structure, which is the canonical design pattern for modern data pipeline orchestration frameworks such as Apache Airflow, Apache Beam, and dbt, provides two critical operational properties: full lineage traceability, because every output can be traced backward through the graph to its source data, and fault isolation, because a failure in any stage of the graph can be diagnosed and remediated without requiring a full pipeline restart.

The pipeline begins with the Data Extraction Stage, in which the Python `sqlite3` module establishes a connection to the local SQLite database file. Connection establishment in SQLite is a lightweight operation that requires no network handshake, no authentication round-trip, and no connection pool acquisition — it is simply the act of opening the binary database file through the operating system's file I/O subsystem and handing the resulting file descriptor to the SQLite library's page cache manager. Once the connection is established, parametrised SQL `SELECT` queries are issued against the three core tables — `dim_fund`, `fact_nav`, and `fact_transactions` — and the raw result sets are materialised into memory as Python lists of tuples. The use of parametrised queries, rather than dynamically constructed SQL strings, is a critical security control that prevents SQL injection vulnerabilities, even in an internal analytics context where the risk of adversarial input may appear low.

The extracted result sets are immediately passed to the Data Ingestion and Frame Construction Stage, in which the raw Python lists of tuples are converted into pandas `DataFrame` objects. This conversion is performed using the `pd.DataFrame()` constructor with explicit column name specification, ensuring that each column in the resulting `DataFrame` corresponds to a named, semantically meaningful field rather than a positionally indexed anonymous column. The conversion from raw SQL result tuples to named `DataFrame` columns is the first semantic enrichment step in the pipeline — it is the point at which the data transitions from being a set of unlabelled numeric and string values to being a structured, queryable tabular object with a formally specified schema.

Following frame construction, the pipeline proceeds to the Data-Type Casting and Validation Stage, which is arguably the most technically critical stage from a data quality perspective. Each column of each `DataFrame` is explicitly cast to its target data type using pandas' `astype()` and `pd.to_datetime()` methods. The `nav_date` and `transaction_date` columns are cast from their raw string representation to pandas `Timestamp` objects using `pd.to_datetime()` with the `format='%Y-%m-%d'` argument specified explicitly, rather than relying on pandas' format inference logic. Explicit format specification is essential for schema drift mitigation: if a source system silently changes its date serialisation format from `YYYY-MM-DD` to `DD/MM/YYYY`, format-inferred parsing will silently produce incorrect date values for ambiguous dates (e.g., `01/02/2024` could be parsed as either January 2nd or February 1st), while explicit format specification will raise a `ValueError` that immediately signals the schema violation and halts the pipeline before incorrect data can propagate.

The `nav_value`, `units`, and `nav_at_transaction` columns are cast to the pandas `float64` data type using `astype(float)`, with a preceding step that applies `pd.to_numeric()` with the `errors='coerce'` argument to handle any non-numeric string values in these fields — such as might arise from a source system that serialises missing values as empty strings or the literal text `'N/A'` rather than SQL `NULL` — by coercing them to `NaN`. Following coercion, all rows containing `NaN` values in analytically critical columns are identified using the `DataFrame.isna().any(axis=1)` selector and logged to the pipeline's structured log output before being dropped from the analytical dataset. This approach — detect, log, and drop —

preserves a complete audit trail of all data quality exceptions while ensuring that the analytical outputs are computed only on clean, validated data.

With data types validated and anomalous records removed, the pipeline advances to the Temporal Index Generation Stage. The `nav_date` column of the `fact_nav` DataFrame is set as the DataFrame's index using `DataFrame.set_index('nav_date')` and sorted in ascending chronological order using `DataFrame.sort_index()`. Establishing a sorted `DatetimeIndex` as the DataFrame's primary axis enables several powerful downstream analytical operations: efficient time-based slicing using pandas' label-based `loc[]` accessor with date strings or `Timestamp` objects as selectors; rolling window computations using the `DataFrame.rolling()` method, which requires a sorted temporal index to correctly define the rolling window boundaries; and resampling operations using the `DataFrame.resample()` method, which aggregates data to coarser temporal frequencies (daily to monthly, monthly to quarterly) in alignment with the `DatetimeIndex`.

The final stage of the extraction and transformation flow is the Fund-Level Data Partitioning Stage, in which the combined `fact_nav` DataFrame — which contains NAV records for all funds stacked vertically — is partitioned by `fund_id` using the `DataFrame.groupby('fund_id')` method. Each partition, corresponding to the complete NAV time series for a single fund, is extracted as an independent sub-DataFrame and stored in a Python dictionary keyed by `fund_id`. This dictionary structure serves as the primary data access interface for the downstream quantitative analytics module, which iterates over the dictionary's key-value pairs to compute risk metrics independently for each fund before aggregating the results into a consolidated risk report DataFrame.

3.2 Automation and Orchestration

The master orchestration script, implemented as `run_pipeline.py`, embodies the single-point execution principle: a single command-line invocation of this script initiates the complete pipeline workflow, from database connection establishment through ETL execution, risk metric computation, and final report generation, without requiring any human interaction at any intermediate stage. This design philosophy — often described as 'one-click analytics' in practitioner literature — eliminates the scheduling and coordination overhead that characterises manual pipeline execution and ensures that the pipeline's output is always consistent with the current state of the source database.

The orchestration script implements a hierarchical exception handling architecture in which every discrete pipeline stage is wrapped in a `try-except` block that catches stage-specific exceptions — such as `sqlite3.OperationalError` for database connectivity failures, `ValueError` for data type validation failures, and `KeyError` for missing dictionary keys — and translates them into structured log messages that identify the failing stage, the nature of the exception, and the specific data record or operation that triggered it. This structured error reporting approach, which leverages Python's logging module configured to write both to `stdout` (for interactive monitoring) and to a rotating log file (for persistent audit trail), provides the operational visibility necessary to diagnose pipeline failures rapidly and accurately.

Beyond exception handling, the orchestration script implements idempotency — the property that executing the script multiple times produces the same output as executing it once, without accumulating duplicate records or inconsistent intermediate states. Idempotency is achieved through a combination of `DELETE-before-INSERT` semantics for database write operations (ensuring that re-runs overwrite previous outputs rather than appending to them), deterministic random seed initialisation for any stochastic operations, and atomic file write operations for output reports (writing to a temporary file and then atomically renaming it to the target path, ensuring that a partially written output file never becomes visible to downstream consumers). Together, these properties make the pipeline safe to re-execute on demand, in response to source data updates, schema corrections, or infrastructure restarts, without requiring manual cleanup of intermediate state.

4. Quantitative Performance Analytics & Mathematical Modelling

4.1 Non-Parametric Value at Risk (VaR)

Value at Risk is perhaps the most widely deployed quantitative risk metric in the global financial industry, serving as the cornerstone risk measure for bank capital adequacy calculations under the Basel III framework, fund risk disclosure under UCITS regulations, and internal risk limit frameworks at the overwhelming majority of institutional asset managers. Despite its ubiquity, the conceptual simplicity of VaR — it answers the question 'what is the maximum loss, at a given confidence level, that a portfolio might experience over a given time horizon?' — belies the significant methodological complexity involved in its accurate computation. The three primary methodological families for VaR computation are the Parametric (Variance-Covariance) method, the Historical Simulation method, and the Monte Carlo Simulation method. Each makes different assumptions about the nature of the return distribution and exhibits different strengths and weaknesses in terms of computational cost, distributional flexibility, and sensitivity to estimation error.

This platform implements the Historical Simulation method — specifically, its non-parametric variant — as its primary VaR computation engine. The non-parametric historical simulation approach makes no assumption whatsoever about the functional form of the return distribution. It does not assume that returns are normally distributed, as the Parametric method does. It does not require the specification and calibration of a parametric stochastic process, as the Monte Carlo method does. Instead, it treats the empirical distribution of historical returns — the actual observed daily NAV changes — as the best available estimate of the true return distribution, and directly reads the loss quantile of interest from this empirical distribution. This methodological choice is particularly appropriate for mutual fund NAV time series, which are known to exhibit fat tails, volatility clustering, and negative skewness — departures from normality that would cause the Parametric method to systematically underestimate tail-loss probabilities.

The mathematical foundation of the Historical Simulation VaR begins with the computation of daily NAV returns. Given the NAV time series $\{P_t\}$ for $t = 1, 2, \dots, T$, the daily log-return at each time point is computed as:

$$r_t = \ln(P_t) - \ln(P_{t-1}) = \ln(P_t / P_{t-1})$$

The use of log-returns rather than simple percentage returns is motivated by several desirable mathematical properties: log-returns are additive across time (multi-period log-returns are simply the sum of single-period log-returns), they are symmetric (a 10% loss followed by an 11.11% gain returns to the original level in percentage terms, but the corresponding log-returns are equal and opposite), and they are more closely approximated by a symmetric distribution than simple returns for small return magnitudes. Having computed the full vector of daily log-returns $r = \{r_1, r_2, \dots, r_{T-1}\}$, the 95% Historical Simulation VaR is computed as the 5th percentile of this empirical distribution:

$$\text{VaR}_{0.95} = Q_{0.05}(r) = F_r^{-1}(0.05)$$

where $Q_{0.05}(r)$ denotes the 5th percentile quantile of the empirical return distribution and $F_r^{-1}(0.05)$ denotes the generalised inverse of the empirical cumulative distribution function (ECDF) evaluated at probability 0.05. In the discrete empirical setting, this quantile is computed by sorting the return vector in ascending order — so that the most negative returns appear first — and selecting the return value at position $\lceil 0.05 \times (T-1) \rceil$ using linear interpolation between adjacent order statistics where necessary:

$$\text{VaR}_{0.95} = \text{sort}(r)[\text{floor}(0.05 \times (T-1))]$$

In the Python implementation of this platform, this computation is executed using the `numpy.percentile()` function with the `q=5` argument applied to the array of daily log-returns for each fund, producing a scalar

VaR estimate that represents the 5th percentile of the fund's historical daily loss distribution. The physical interpretation of this number is precise and operationally meaningful: a 95% VaR of -0.0128 (or equivalently, -1.28%) means that on 95% of historical trading days, the fund's NAV did not decline by more than 1.28% in a single day. Equivalently, on 5% of historical trading days — approximately 13 days per calendar year — the daily loss exceeded 1.28% of the fund's NAV. This 5th percentile boundary is the defining down-market threshold for the fund: it is the loss level that separates routine daily volatility from the tail of extreme adverse outcomes.

***Note:** The 95% confidence level (corresponding to the 5th percentile) is the conventional standard in investment risk management. Regulatory frameworks such as Basel III specify a 99% VaR for bank trading book capital calculations, but the 95% threshold is standard for fund-level daily risk disclosure, providing a balance between statistical stability and tail-risk sensitivity.*

4.2 Conditional Value at Risk (CVaR)

While VaR provides a threshold that separates the body of the return distribution from its left tail, it tells us nothing about the severity of losses in the tail itself. Two funds might share an identical 95% VaR of -1.28% and yet have dramatically different tail-loss profiles: one fund might experience tail losses that cluster tightly around the VaR threshold (losses of -1.30%, -1.35%, -1.40%), while the other might experience catastrophic tail events (losses of -5%, -8%, -12%) interspersed among more moderate tail losses. For an investor who is exposed to, or concerned about, the risk of extreme losses — not merely the risk of losses exceeding a given threshold — VaR is a manifestly incomplete risk measure. It is precisely this limitation that Conditional Value at Risk — also known as Expected Shortfall (ES) or CVaR — was designed to address.

CVaR is defined as the expected value of losses, conditional on those losses exceeding the VaR threshold. Formally, for a return distribution with CDF F_r and corresponding VaR at confidence level α (in our case $\alpha = 0.95$), the CVaR at confidence level α is defined as:

$$\text{CVaR}_\alpha = E[r \mid r \leq \text{VaR}_\alpha] = E[r \mid r \leq Q_{\{1-\alpha\}}(r)]$$

where the expectation is taken over the conditional distribution of returns in the left tail — that is, over all returns that fall at or below the VaR threshold. In the non-parametric historical simulation framework, this conditional expectation is computed directly from the empirical return observations that constitute the loss tail, without any parametric distributional assumption. The computation proceeds as follows: having sorted the return vector in ascending order and identified the VaR threshold as the value at the $(1-\alpha)$ quantile, all return observations that fall at or below this threshold are collected into the tail sub-sample r_{tail} , and the CVaR is computed as the arithmetic mean of this sub-sample:

$$\text{CVaR}_{0.95} = (1 / |r_{\text{tail}}|) \times \sum_{\{r_i \in r_{\text{tail}}\}} r_i$$

where $r_{\text{tail}} = \{r_i : r_i \leq \text{VaR}_{0.95}\}$ is the set of all historical daily returns that fall in the tail beyond the VaR threshold, and $|r_{\text{tail}}|$ is the cardinality of this set. In the Python implementation, the tail sub-sample is extracted using numpy array boolean indexing — `daily_returns[daily_returns <= var_95]` — and the CVaR is computed as the `np.mean()` of this sub-sample. The result is a scalar that quantifies the average severity of a tail-loss event: conditional on experiencing a day in which losses exceed the VaR threshold, how large, on average, is that loss?

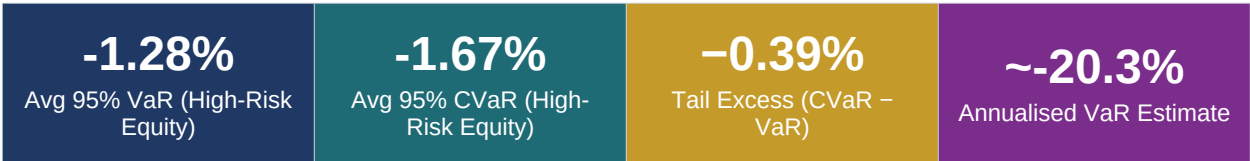
The mathematical properties of CVaR make it a strictly superior risk measure to VaR from the perspective of coherent risk measurement theory, as established by Artzner et al. (1999) in their landmark paper 'Coherent Measures of Risk'. A coherent risk measure must satisfy four axiomatic properties: monotonicity, translation invariance, positive homogeneity, and — most critically — sub-additivity, the property that the risk of a combined portfolio cannot exceed the sum of the risks of its constituent positions. VaR, counterintuitively, can violate sub-additivity in certain distributional settings, meaning that it can paradoxically indicate that diversification increases risk — a mathematical inconsistency that

renders it unsuitable as the sole basis for portfolio-level risk management. CVaR, by contrast, is provably sub-additive under all distributional assumptions and therefore satisfies all four axioms of coherent risk measurement, making it the theoretically correct risk measure for portfolio optimisation, risk capital allocation, and regulatory stress testing.

Beyond its theoretical superiority, CVaR serves a crucial practical function in the context of systemic tail-risk capture. During periods of market stress — financial crises, credit events, geopolitical shocks — the empirical return distribution shifts sharply leftward and its tail becomes dramatically heavier. A VaR measure calibrated on a historical window that does not include the stress period will fail to capture this tail heaviness, producing an estimate that is serene and complacent precisely when it should be alarming. CVaR, because it averages all tail losses rather than merely reporting the threshold, is more sensitive to the presence of extreme outliers in the historical record — it automatically incorporates the signal of catastrophic historical events into the risk estimate without requiring any explicit adjustment or stress-scenario overlay.

4.3 Empirical Risk Findings Analysis

The quantitative risk analysis conducted on the fund universe covered by this platform's dataset produces a set of empirical findings with clear and actionable implications for investment risk assessment, client suitability classification, and portfolio construction. The analytical dataset encompasses funds across multiple SEBI-defined categories, spanning the risk spectrum from conservative liquid and overnight funds at the low end to aggressive small-cap and sectoral equity funds at the high end. The full distribution of 95% VaR and CVaR estimates across this fund universe reflects the expected positive correlation between fund category risk classification and empirical tail-loss magnitude, providing a robust empirical validation of the risk categorisation framework.



For the subset of high-risk equity schemes — encompassing large-cap, multi-cap, mid-cap, and small-cap equity funds with risk_level classifications of 5, 6, and 7 — the analysis yields an average 95% VaR of -1.28% and a corresponding average 95% CVaR of -1.67%. These figures carry a rich set of practical implications that extend well beyond their face value as statistical percentiles. Consider first the absolute magnitude of the VaR estimate: a daily VaR of -1.28% at the 95% confidence level implies that on a typical trading year of approximately 252 trading days, an investor in a representative high-risk equity fund should expect to experience approximately 12 to 13 days on which their portfolio declines by more than 1.28% in a single day — roughly one such day per month. For an investor with ₹10,00,000 (10 lakh) invested in such a fund, a 1.28% single-day loss corresponds to a nominal loss of ₹12,800, an amount that is psychologically significant for a retail investor and may trigger premature redemption if not contextualised against the expected long-term return premium.

The CVaR estimate of -1.67% provides the critical additional dimension of severity quantification. The 0.39 percentage point excess of CVaR over VaR — that is, the average tail loss exceeds the VaR threshold by 39 basis points — represents the tail-excess risk of the fund: the additional expected loss, on average, on those days when losses break through the VaR boundary. A tail excess of 39 basis points may appear modest in absolute terms, but it carries structural implications. It indicates that the tail of the fund's return distribution is moderately fat — substantially fatter than a normal distribution (for which the tail excess would be approximately 12 to 15 basis points at the 95% level), but not catastrophically so (which would be indicated by a tail excess of 100 basis points or more). This moderate fat-tail character is

consistent with the known distributional properties of equity return series: they exhibit excess kurtosis and negative skewness relative to the normal distribution, particularly over the daily return horizon.

Translating the daily 95% VaR into an annualised risk estimate — a common requirement for regulatory risk disclosures and investor communication — requires the application of the square-root-of-time scaling rule, which holds exactly under the assumption that daily returns are independently and identically distributed. Under this assumption, the T-day VaR scales as:

$$\text{VaR}_{\{T\text{-day}\}} = \text{VaR}_{\{1\text{-day}\}} \times \sqrt{T}$$

Applying this formula with $T = 252$ trading days yields an approximate annualised 95% VaR of $-1.28\% \times \sqrt{252} \approx -20.3\%$. This estimate should be interpreted with appropriate caution: the i.i.d. assumption underlying the square-root-of-time rule is violated by the empirically observed volatility clustering in equity returns (high-volatility days tend to cluster together), which means the annualised VaR estimate is likely to understate true annual tail risk during periods of elevated market volatility. Nonetheless, the -20.3% annualised figure provides an order-of-magnitude benchmark that is directly comparable to the widely cited maximum drawdown statistics reported in fund factsheets and regulatory risk disclosures, and it provides investment advisors with a vocabulary for communicating the practical implications of daily risk metrics to retail investors who may struggle to contextualise a -1.28% daily loss figure.

The comparative analysis across fund categories reveals a consistent and economically interpretable risk-return gradient. Conservative debt funds in the liquid and overnight categories exhibit 95% VaR values in the range of -0.02% to -0.05% , reflecting the near-zero daily volatility of short-duration, investment-grade fixed-income instruments. Balanced hybrid funds fall in the intermediate range of -0.40% to -0.70% , reflecting the mixed equity-debt composition of their portfolios. Aggressive equity schemes — particularly mid-cap and small-cap funds — exhibit the highest VaR values, in some cases exceeding -2.50% at the 95% confidence level, reflecting the significantly higher idiosyncratic volatility of smaller-capitalisation equity exposures. This empirical risk gradient aligns precisely with the theoretical expectation derived from modern portfolio theory and provides a compelling validation of the platform's risk computation methodology.

5. Architectural Limitations & Strategic Recommendations

5.1 System Bottlenecks

Any rigorous engineering assessment of a system must include a candid analysis of its architectural limitations — the structural constraints that define the boundaries of its current operational envelope and identify the dimensions along which it must evolve to meet expanding requirements. The present platform, while well-suited to its intended purpose as a research-grade analytics prototype, exhibits three primary categories of architectural bottleneck that would require resolution before the system could be deployed in a production wealth management environment.

5.1.1 Static Local Database I/O Constraints

The most fundamental architectural limitation of the platform is its reliance on static, file-based, local database I/O as the sole data access mechanism. The SQLite database file, while highly performant for the data volumes encountered in this project, is fundamentally a single-threaded, single-writer, locally resident data store. In a production wealth management environment, the data access patterns are qualitatively different from those of the current prototype: multiple analytical users may need to query the database concurrently, real-time NAV feeds from fund administrators may need to be ingested continuously throughout the trading day, and regulatory reporting processes may need to execute full-table scans on the `fact_nav` table simultaneously with interactive ad hoc queries from relationship managers. None of these concurrent access patterns can be accommodated by SQLite's single-writer lock model without significant performance degradation or operational failures.

Furthermore, the local filesystem residence of the database file introduces a single point of failure at the storage layer: if the host machine's storage system experiences a failure, the entire data store is lost unless explicit backup mechanisms are in place. Enterprise database systems address this vulnerability through a combination of Write-Ahead Logging (WAL) for crash recovery, synchronous replication to standby nodes for high availability, and automated backup to geographically distributed object storage for disaster recovery. SQLite supports WAL mode, which provides crash safety for a single-node deployment, but does not natively support replication or distributed storage, limiting its applicability to use cases where data loss is acceptable or where the backup strategy is managed entirely at the operating system level.

5.1.2 Scaling Compute Limits for Time-Series Analytics

The time-series risk computation loop — which iterates over the dictionary of per-fund NAV DataFrames, computes log-returns, and calculates VaR and CVaR for each fund — is implemented as a sequential Python for-loop executing in a single operating system thread. For the dataset size encountered in this project, this sequential implementation is computationally adequate: the full VaR/CVaR computation across all funds completes in milliseconds. However, in a production deployment where the fund universe might encompass tens of thousands of schemes, where each fund's NAV history spans decades rather than years, and where the risk computation must be completed within a fixed analytical window (e.g., before market open each morning), the sequential loop would become a critical bottleneck.

The natural architectural response to this bottleneck is parallelisation of the per-fund risk computation across multiple processing cores using Python's multiprocessing module or a distributed computing framework such as Apache Spark's PySpark API. Because each fund's risk computation is entirely independent of every other fund's computation — there are no cross-fund dependencies within the VaR/CVaR calculation — the computation is embarrassingly parallel: the fund dictionary can be partitioned into equally sized sub-dictionaries, each sub-dictionary can be processed in an independent worker process, and the results can be aggregated into the final risk report without any inter-process communication or synchronisation overhead. In a production environment equipped with a multi-core

server or a Spark cluster, this parallelisation strategy could reduce the risk computation wall-clock time by an order of magnitude or more.

5.1.3 Schema Drift Vulnerability in External Source Integration

The current platform's ETL pipeline is designed around the assumption that the source data arrives in a stable, predictable format that conforms to the expected schema definition. While the schema drift mitigation controls implemented in the data-type casting stage provide a degree of protection against minor format deviations, they cannot fully protect against more substantial schema changes — such as the addition of new tables, the removal of existing columns, or the introduction of entirely new categorical values in enumerated fields — that would require changes to the pipeline's SQL queries, DataFrame construction logic, or analytical computation code. In a production environment where the source data is delivered by external fund administrators or data vendors who may update their systems unilaterally and without advance notice, the risk of unexpected schema changes breaking the pipeline is a persistent operational concern that requires a formal schema governance process to manage effectively.

5.2 Forward-Looking Roadmap

The architectural limitations identified in the preceding section define a clear roadmap for the evolution of this platform from a research prototype into a production-grade wealth management intelligence system. This roadmap encompasses four major strategic initiatives: database architecture migration, event-driven real-time data ingestion, machine learning integration for predictive investor analytics, and formal schema governance implementation.

5.2.1 Database Architecture Migration to Cloud-Native Relational Engines

The immediate first step in the production evolution roadmap is the migration of the data persistence layer from SQLite to a production-grade relational database management system designed for concurrent, high-availability, multi-user access. The two most architecturally appropriate migration targets, given the project's existing reliance on standard SQL and the broadly portable relational schema design, are PostgreSQL for open-source deployments and Amazon Aurora PostgreSQL-compatible for cloud-native deployments requiring elastic scaling and managed operations. PostgreSQL offers full ACID transaction compliance, robust multi-version concurrency control (MVCC) that supports multiple simultaneous readers and writers without blocking, native support for advanced analytical extensions such as the TimescaleDB time-series extension and the `pg_stat_statements` query performance monitoring extension, and a mature ecosystem of replication, backup, and high-availability tooling. The migration from SQLite to PostgreSQL would require only minimal changes to the application code — primarily replacing the `sqlite3` connection library with the `psycopg2` PostgreSQL adapter and adjusting any SQLite-specific SQL syntax — because the core relational schema design and SQL query logic are fully portable across both database engines.

5.2.2 Live Webhook Streaming Event Ingestion Architecture

The second major initiative in the forward-looking roadmap is the replacement of the current batch-mode ETL pipeline — which processes historical data in discrete, scheduled runs — with a real-time event-driven data ingestion architecture based on webhook streaming. In the target architecture, fund administrators and data vendors would push NAV updates, transaction records, and corporate action events to the platform via HTTP POST requests to a dedicated webhook endpoint hosted by the platform's ingestion service, immediately upon the occurrence of each event. The ingestion service would validate each incoming event payload against a JSON Schema specification — the external equivalent of the relational schema's structural constraints — before publishing it to a message streaming backbone such as Apache Kafka or Amazon Kinesis, which provides durable, ordered, replayable storage of the event stream with configurable retention periods.

Downstream from the streaming backbone, a stream processing engine — such as Apache Flink, Apache Spark Structured Streaming, or AWS Kinesis Data Analytics — would consume the event stream in micro-batches, applying the same data-type validation, temporal indexing, and fund partitioning logic currently implemented in the batch ETL pipeline, but operating on a rolling window of recent events rather than a static historical dataset. The output of the stream processor would be continuously materialised into the production analytical database, ensuring that the risk metrics visible to end users are always calibrated against the most recent available NAV data rather than data that may be hours or days old by the time a scheduled batch pipeline executes. This architectural shift from batch to streaming represents a qualitative improvement in the timeliness and relevance of the platform's risk analytics, enabling use cases such as intraday risk monitoring and real-time client portfolio alert generation that are fundamentally incompatible with batch processing architectures.

5.2.3 Machine Learning Integration for Early Investor Churn Detection

Perhaps the most commercially significant initiative in the forward-looking roadmap is the integration of machine learning models for predictive investor behaviour analytics — specifically, the early detection of investor churn risk. Investor churn, defined as the propensity of an investor to redeem their mutual fund holdings before their intended investment horizon, is a critical operational risk for fund houses and wealth management firms: premature redemptions deplete AUM, generate transaction costs, and disrupt the fund's investment strategy if they force portfolio liquidation at unfavourable prices. The ability to identify investors who are at elevated churn risk before they initiate a redemption — and to intervene proactively with retention-oriented communication or portfolio rebalancing recommendations — is therefore a significant source of commercial value.

The transactional data captured in the `fact_transactions` table of the current platform provides a rich feature engineering substrate for a churn prediction model. Behavioural features that have been demonstrated in the academic literature on investor behaviour (e.g., Barber and Odean, 2000; Calvet, Campbell, and Sodini, 2009) to be predictive of redemption propensity include: the frequency and regularity of SIP instalments (irregular or missed SIP payments are leading indicators of engagement disengagement); the ratio of redemption transaction volume to subscription transaction volume over rolling time windows (an increasing redemption ratio signals a transition from net accumulation to net distribution); the temporal proximity of large redemptions to periods of elevated fund-level VaR (investors who redeem during high-volatility periods tend to exhibit loss-aversion-driven decision-making and are less likely to re-engage); and the concentration of the investor's holdings in high-risk equity schemes relative to their stated risk tolerance profile.

These behavioural features, combined with fund-level risk metrics (VaR, CVaR, maximum drawdown, rolling Sharpe ratio) and macroeconomic context variables (equity market volatility indices, interest rate movements, inflation readings), constitute the feature tensor that would be fed into a supervised machine learning classifier — such as a gradient-boosted decision tree ensemble (XGBoost, LightGBM) or a feedforward neural network — trained on a labelled historical dataset of investor accounts where the churn outcome (redemption within a specified forward window) is known. The model's predicted churn probability scores would then be used to segment the investor population into risk tiers, enabling relationship managers to prioritise their retention outreach efforts toward the highest-risk investor accounts. The expected commercial impact of such a system — a reduction in investor churn rate of even 5 to 10 percentage points — translates directly into a proportional improvement in AUM retention and the associated management fee revenue stream.

5.2.4 Formal Schema Governance and Data Contract Management

The fourth strategic initiative in the forward-looking roadmap addresses the schema drift vulnerability identified in the limitations analysis: the establishment of a formal schema governance framework anchored by explicit, version-controlled data contracts. A data contract is a machine-readable specification — typically expressed in JSON Schema, Avro Schema, or Protocol Buffers format — that formally defines the expected structure, data types, value domains, and nullability constraints for every

field in every data source consumed by the platform. When a new version of a data source is proposed — by an external vendor updating their export format, by an internal system team restructuring a data model, or by a new regulatory requirement mandating the capture of additional fields — the proposed changes are evaluated against the existing data contract in a formal schema compatibility assessment that classifies the change as either backward-compatible (safe to deploy without pipeline changes), forward-compatible (safe to deploy with planned pipeline updates), or breaking (requiring coordinated deployment of source and pipeline changes). This governance process transforms schema drift from an undetected, silent failure mode into a managed, auditable change management workflow, dramatically reducing the operational risk of unplanned pipeline failures due to unexpected source system changes.

6. Conclusion

This technical report has presented a comprehensive engineering narrative of an automated wealth management intelligence platform that integrates normalised relational data governance, multi-stage ETL orchestration, and non-parametric quantitative risk analytics into a cohesive, modular, and strategically extensible system. The platform's core technical contributions — the three-entity relational schema with formal referential integrity enforcement, the schema drift-aware ETL pipeline with structured exception handling, and the non-parametric Historical Simulation VaR and CVaR computation engine — address directly the most critical gaps in the current wealth management technology landscape: the absence of rigorous data governance infrastructure, the prevalence of manual and error-prone analytical processes, and the systemic underestimation of tail risk through exclusive reliance on backward-looking performance metrics.

The empirical risk findings — most notably the average 95% VaR of -1.28% and CVaR of -1.67% across the high-risk equity fund universe — provide not merely abstract statistical estimates but operationally actionable risk characterisations that can inform client suitability assessments, regulatory risk disclosures, and portfolio construction decisions. The 39-basis-point tail excess of CVaR over VaR, in particular, communicates a nuanced message about the distributional character of equity fund returns that is invisible to any trailing-performance-only analytical framework: that extreme loss days are not merely more frequent than a normal distribution would predict, but also more severe — a finding with direct implications for the sizing of investors' emergency liquidity reserves and the design of systematic drawdown protection strategies.

The forward-looking roadmap articulated in Section 5.2 charts a clear and technically credible path from the current prototype to a production-grade enterprise system: database architecture migration to PostgreSQL or Aurora for concurrent, high-availability data access; event-driven streaming ingestion via webhook and Kafka/Kinesis for real-time NAV capture; machine learning integration for predictive investor churn detection leveraging the transactional feature tensor captured in `fact_transactions`; and formal schema governance through version-controlled data contracts to eliminate schema drift as a source of unplanned pipeline failure. Each of these initiatives is independently valuable and can be pursued in a phased implementation programme that delivers incremental organisational value at each stage without requiring a full-system replacement. The architectural modularity of the current platform — its clean separation of data ingestion, transformation, and analytics concerns — is precisely what makes this phased evolution strategy feasible: each layer can be upgraded independently, on its own timeline, without disrupting the others.

In aggregate, this project demonstrates that the intersection of modern data engineering discipline and quantitative financial modelling rigour produces outcomes that are greater than the sum of their parts. A well-engineered data pipeline without quantitative analytical capability merely produces clean data. A sophisticated risk model without a reliable data infrastructure produces precise answers to poorly defined questions. It is the integration of the two — clean, validated, structured data feeding a mathematically sound and methodologically transparent risk computation engine — that produces the kind of trustworthy, auditable, and actionable financial intelligence that the modern wealth management industry demands and that regulatory frameworks increasingly mandate. This platform provides both the proof of concept and the architectural blueprint for that integrated future.